

Exercise 1.1 – 1.4

Ex. 1.1 (5 pts) — Magic number of a binary executable

Role. A magic number is a short, fixed byte pattern that a file carries so that its *format* can be recognised from its contents rather than from its name. When the kernel services an `execve()`, it does not trust the file extension: it reads the first handful of bytes and matches them against the formats it knows, and that match is what selects the right loader (native ELF loader, script interpreter, emulation layer, ...). If nothing matches, the call is simply refused with `ENOEXEC` instead of the machine jumping into arbitrary bytes. It therefore does two jobs at once: format dispatch and a cheap integrity/sanity check.

Where it is stored. In the very first bytes of the file, at offset 0, as the opening field of the file header — which is exactly what makes the check cheap, since the kernel already has to read that first block anyway. Examples: an ELF binary starts with `0x7F 'E' 'L' 'F'` (the first four bytes of `e_ident` in the ELF header); a shell or Python script starts with `#! (0x23 0x21)` followed by the interpreter path; a Windows PE image starts with `'M' 'Z'`; a Java class file starts with `0xCAFEBAFE`.

Ex. 1.2 (5 pts) — Segments of an address space

A process address space is built from the following regions, conventionally laid out from low addresses to high:

- **Text (code) segment** — the machine instructions of the program. Mapped read-only and executable, so it can be shared by every process running the same binary and cannot be scribbled over by a stray pointer. Fixed size.
- **Initialized data segment (.data)** — global and static variables whose initial value is not zero. Their values are stored in the executable file and copied in by the loader. Read/write, fixed size.
- **Uninitialized data segment (BSS)** — global and static variables that start out zero. Only their *size* is recorded in the file, not their contents; the loader zero-fills the region, so the BSS costs nothing on disk. Read/write, fixed size.
- **Heap** — memory obtained at run time by `malloc/new`. It sits above the BSS and **grows upward**, toward higher addresses, as the allocator extends it (`brk/sbrk`, or `mmap` for large requests).
- **Memory-mapped region** — shared libraries, files mapped with `mmap`, and large allocations. It occupies the middle ground between heap and stack, and is how dynamically linked code is brought into the space.
- **Stack** — one per thread; holds activation records: parameters, local variables, saved registers and return addresses. It is placed at the top of the space and **grows downward**, toward lower addresses, one frame per call.

The heap and the stack thus grow toward each other, and the unmapped gap between them is what keeps them apart; a program that runs off either end walks into that hole, which is precisely the situation of Ex. 1.4.

Ex. 1.3 (5 pts) — Von Neumann machines and machines today

A von Neumann machine is the *stored-program* design: a CPU (control unit plus ALU and registers), a single main memory that holds instructions *and* data indistinguishably, and I/O, all joined by a shared bus. Its defining idea is that the program is itself just data sitting in memory, so the

machine is general purpose — you change what it does by loading different bytes, not by rewiring it — and it operates by repeating the fetch–decode–execute cycle, with a program counter naming the next instruction. The price of the single shared memory and bus is that instruction fetch and data access compete for the same path, the well-known *von Neumann bottleneck*.

Today’s computers are still von Neumann machines at the level the programmer sees: x86, ARM and RISC-V all present one flat address space containing both code and data, a program counter, and sequential instruction semantics — which is why a compiled program can be written into memory and then jumped to at all. What has changed is everything underneath, added mostly to hide that bottleneck: split L1 instruction and data caches (a Harvard-style separation smuggled inside a von Neumann machine), deep pipelines, superscalar and out-of-order execution, branch prediction, and many cores sharing memory. So the model is inherited essentially unchanged; only the implementation has become elaborate.

Ex. 1.4 (5 pts) — “Segmentation fault (core dumped)”

What the message means. The program touched memory it was not entitled to touch. The MMU checks every address as it is translated, and when the address is either not mapped at all or is mapped with the wrong permission (writing a read-only page, executing a non-executable one), it raises a hardware fault. The kernel finds no legitimate reason for the fault, converts it into the signal `SIGSEGV`, and delivers it to the process; the default action for `SIGSEGV` is to kill the process and write out a core dump. The line you see is the shell afterwards reporting how its child died.

Possible reasons.

- Dereferencing a null pointer (typically an unchecked return from `malloc` or `fopen`).
- Dereferencing an uninitialized or otherwise wild pointer.
- Use-after-free, or following a dangling pointer to a stack frame that has already returned.
- Running past the end of an array or buffer, far enough to leave the mapping.
- Writing through a `char *` that points at a string literal, which lives on a read-only page.
- Unbounded recursion overflowing the stack into the guard page below it.
- Corrupting the allocator’s metadata (double free, overrun of a heap block), so that a later allocation returns nonsense.

What “segmentation” means here. The word is historical. On machines with segmented memory the address space was divided into segments, each with a base, a limit and access rights, and the hardware trapped any reference that fell outside the bounds or rights of the current segment — a *segmentation violation*. Mainstream systems are paged rather than segmented now, but the name stuck, and it means what it always meant: an access outside a valid, permitted region of the address space.

What “core” means here. “Core” is magnetic-core memory, the ferrite-ring main memory of machines of the 1950s and 60s; “core” became a synonym for main memory and outlived the technology. A core dump is therefore an image of the process’s memory — its segments as listed in Ex. 1.2 — together with its register state, captured at the instant it faulted and written to a file so that a debugger can be pointed at it afterwards and reconstruct where and why the program died.